

Proyecto de Curso: Análisis, Diseño y Construcción de un Sistema Operativo desde Cero

Por

Castro Pari, Rayneld Fidel

Mamani Flores, Natan

Mendoza Quispe, Jose Daniel

Polo Chura, Marco Rosauro

Trabajo académico presentado a la

Facultad de Ingeniería Eléctrica, Electrónica, Informática y Mecánica

como

Proyecto de Investigación de la Primera Unidad de Sistemas Operativos



Departamento Académico de Ingeniería de Informática y de Sistemas

Asesor: Ugarte Rojas, Héctor Eduardo

Memorial Universidad Nacional San Antonio Abad del Cusco

Semestre 2025-II

Cusco

Perú

Índice general

Índice de tablas	iv
Índice de figuras	v
Introducción	1
1 Propuestas analizadas	4
1.1 RedoxOS	5
1.1.1 Nombre del proyecto o sistema operativo	5
1.1.2 Enlace al repositorio y/o documentación oficial	5
1.1.3 Objetivo del proyecto	5
1.1.4 Lenguaje(s) de implementación	6
1.1.5 Arquitectura del sistema	6
1.1.6 Componentes implementados	7
1.1.7 Herramientas utilizadas	11
1.1.8 Nivel de complejidad y accesibilidad para estudiantes	12
1.2 FlexOS	14
1.2.1 Nombre del proyecto o sistema operativo	14

1.2.2	Enlace al repositorio y/o documentación oficial	14
1.2.3	Objetivo del proyecto	14
1.2.4	Lenguaje(s) de implementación	15
1.2.5	Arquitectura del sistema	15
1.2.6	Componentes implementados	16
1.2.7	Herramientas utilizadas (compiladores, emuladores, etc.) . . .	20
1.2.8	Nivel de complejidad y accesibilidad para estudiantes	21
1.3	FreeDOS	22
1.3.1	Nombre del proyecto o sistema operativo	22
1.3.2	Enlace al repositorio y/o documentación oficial	22
1.3.3	Objetivo del proyecto	22
1.3.4	Lenguaje(s) de implementación	22
1.3.5	Arquitectura del sistema	23
2	Comparación técnica	24
3	Análisis crítico	25
	Referencias	26

Índice de tablas

1.1	Funcionamiento técnico de los principales subsistemas de Redox OS .	8
1.2	Funcionamiento técnico de los principales subsistemas de FlexOS . .	18

Índice de figuras

1.1	Comparativa de la arquitectura de Redox frente a arquitecturas tradicionales.	7
1.2	Arquitectura modular e híbrida de FlexOS.	16
1.3	Arquitectura clásica de MS-DOS, base de FreeDOS.	23

Introduccion

El desarrollo y la transformación de los sistemas operativos han sido fundamentales en la evolución de las tecnologías de la información. Desde los primeros programas creados para las computadoras centrales hasta los sistemas más utilizados en ordenadores personales, dispositivos móviles y equipos embebidos, los sistemas operativos han actuado como un puente entre las necesidades del usuario y la complejidad del hardware. Estudiarlos no solo permite entender el funcionamiento básico de una computadora, sino que también brinda las bases conceptuales y técnicas necesarias para el diseño de nuevas soluciones informáticas. Este documento tiene como finalidad ofrecer una perspectiva global de los sistemas operativos, abordando sus características principales, su arquitectura y sus componentes.

Objetivo de la investigacion

El objetivo principal de este trabajo es realizar un análisis técnico y comparativo de distintos sistemas operativos, con la finalidad de identificar sus características más relevantes y examinar las diversas concepciones y enfoques aplicados en su diseño e implementación. Se estudiarán tanto arquitecturas tradicionales, como la monolítica,

así como modelos más modernos, entre ellos los exokernels, híbridos y microkernels . Del mismo modo, se pretende ofrecer una visión crítica sobre las ventajas, limitaciones y ámbitos de aplicación de cada sistema operativo.

Alcance del análisis

El presente estudio se centra en sistemas operativos ampliamente utilizados y representativos de diferentes enfoques arquitectónicos y áreas de aplicación. No se abordarán sistemas altamente experimentales o versiones obsoletas a menos que sean relevantes para la comparación histórica o conceptual. El análisis incluye tanto sistemas de propósito general (Windows, macOS, Linux) como sistemas especializados (RTOS, embebidos y móviles), enfocándose en aspectos técnicos, operativos y comunitarios que permitan identificar fortalezas, limitaciones y escenarios de uso recomendados para cada sistema operativo.

Metodologia

Con el fin de alcanzar los objetivos planteados, se seguirá una metodología compuesta por tres fases fundamentales:

1. **Revisión bibliográfica y documental:** Se consultarán libros especializados, artículos académicos, blogs técnicos, documentación oficial y repositorios relacionados con los sistemas operativos en estudio.
2. **Evaluación técnica:** Se examinarán los aspectos esenciales de cada sistema, incluyendo la gestión de procesos, la administración de memoria, los sistemas de

archivos, el manejo de dispositivos, las interfaces de usuario y los mecanismos de seguridad.

3. **Comparación estructurada:** Se elaborará una tabla que sintetice las características más representativas de los sistemas operativos analizados, tomando en cuenta factores como el tamaño del kernel, la arquitectura, la comunidad, el lenguaje de implementación, la documentación disponible y el soporte de hardware.

Este enfoque permitirá garantizar un análisis más objetivo, completo y útil, tanto para fines académicos como prácticos.

Capítulo 1

Propuestas analizadas

1.1 RedoxOS

1.1.1 Nombre del proyecto o sistema operativo

El sistema operativo examinado es "Redox OS". De acuerdo con (Redox OS Project, 2025c), su denominación deriva de la palabra "Redox", que es la abreviatura de "*reduction-oxidation*", un término de química que representa el intercambio equilibrado de electrones; esto refleja la meta del proyecto: alcanzar un sistema equilibrado en seguridad, rendimiento y simplicidad, a través de un diseño contemporáneo y fiable, redactado totalmente en Rust.

1.1.2 Enlace al repositorio y/o documentación oficial

- Enlace al repositorio oficial: <https://github.com/redox-os>
- Enlace al libro oficial: <https://doc.redox-os.org/book/>

1.1.3 Objetivo del proyecto

Según (Redox OS Project, 2025c,g), Redox OS tiene un enfoque fundamentalmente educativo y experimental, pero también pretende ofrecer una opción práctica y segura en comparación con sistemas Unix tradicionales. Su objetivo es probar que un sistema operativo que esté totalmente desarrollado en Rust puede lograr la misma estabilidad y eficiencia que los sistemas comerciales, mientras elimina categorías completas de errores relacionados con la memoria y la concurrencia.

1.1.4 Lenguaje(s) de implementación

Redox OS está predominantemente desarrollado en Rust, abarcando su kernel, controladores, gestor de memoria, sistema de archivos y gran parte de las herramientas de usuario.

Ciertos elementos específicos, como el cargador de arranque y aspectos de la compatibilidad con C, emplean "ensamblador y C" de manera restringida.

1.1.5 Arquitectura del sistema

Redox OS adopta una arquitectura híbrida de microkernel, inspirada en el diseño de Minix 3 y el modelo de Unix, pero con un enfoque mucho más fuerte en la seguridad y la modularidad. El kernel de Redox es extremadamente pequeño: se encarga solo de la planificación, la gestión básica de memoria y la comunicación entre procesos. Todos los servicios del sistema, como el manejo de archivos, controladores, red y entorno gráfico, se ejecutan en el espacio de usuario como procesos independientes llamados schemes.

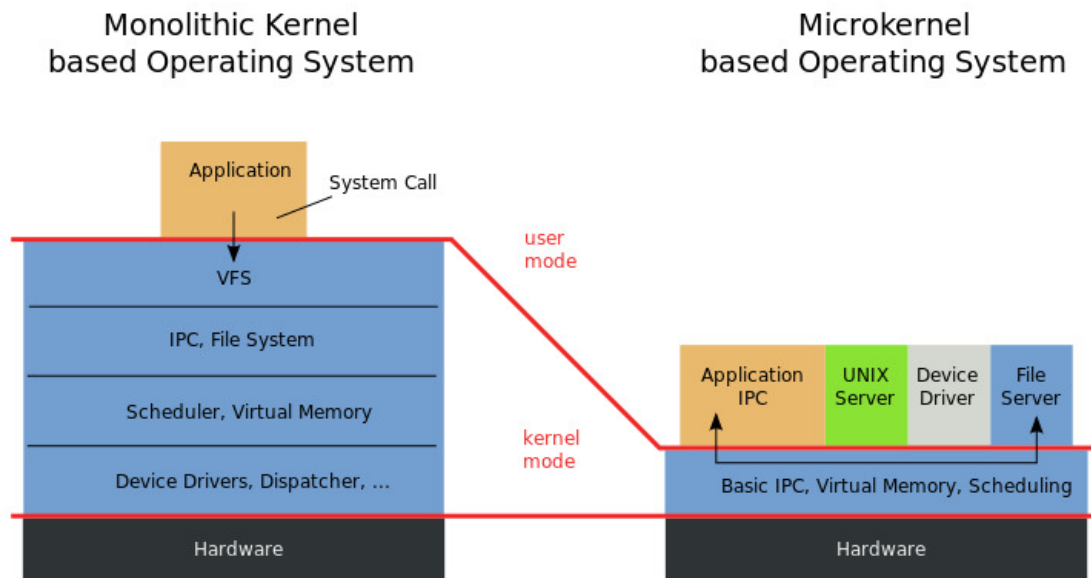


Figura 1.1: Comparativa de la arquitectura de Redox frente a arquitecturas tradicionales.

Fuente: Obtenido de (Redox OS Project, 2025c).

1.1.6 Componentes implementados

En Redox OS, los componentes principales del sistema se implementan como crates escritos en **Rust**, siguiendo una arquitectura microkernel. Cada crate encapsula un servicio o subsistema (procesos, memoria, sistema de archivos, E/S, etc.) con una estricta separación de privilegios. El microkernel se encarga de la comunicación por mensajes entre estos servicios, garantizando seguridad, aislamiento y estabilidad.

A continuación, se presenta una tabla con los principales componentes y subsistemas de Redox OS, los nombres de sus crates en el repositorio oficial y la idea técnica detrás de su funcionamiento.

Tabla 1.1: Funcionamiento técnico de los principales subsistemas de Redox OS

Componente / Sub-sistema	Nombre en Redox (archivo/crate)	Funcionamiento técnico (idea clave)
Gestión de memoria	<code>memory</code> , <code>paging</code> , <code>rmm</code>	Maneja paginación virtual y asignación segura de marcos físicos.
Gestión de procesos y multitarea	<code>kernel</code> , <code>scheme::proc</code>	Ejecuta procesos aislados con planificación cooperativa.
Sistema de archivos	<code>redoxfs</code>	Sistema nativo en Rust con soporte concurrente.
Subsistemas de E/S y drivers	<code>drivers</code> , <code>schemes</code>	Drivers en espacio de usuario con namespaces restringidos.
Comunicación entre procesos (IPC)	<code>syscall</code> , <code>scheme</code> , <code>libredox</code>	IPC basado en mensajes mediante estructuras SQE/CQE.
Planificación y temporización	<code>scheduler</code> , <code>timer</code>	Planificador por prioridad con temporizador interno.
Gestión de usuarios y permisos	<code>user</code> , <code>authd</code>	Controla autenticación y separación de privilegios.
Interfaz gráfica (GUI)	<code>orbital</code> , <code>orbtk</code>	Servidor de ventanas y toolkit gráfico modular.
Red y conectividad	<code>netstack</code> , <code>ethernetd</code> , <code>ipd</code>	Pila de red completa en espacio de usuario.
Gestión del sistema y servicios	<code>init</code> , <code>logd</code> , <code>backgroundd</code>	Controla inicio, logs y servicios del sistema.
Bibliotecas del sistema y compatibilidad	<code>relibc</code> , <code>redox_rt</code> 8	Implementan llamadas POSIX sobre los <i>schemes</i> .

Fuente: Elaboración propia con base en (Redox OS Project, 2025g,c).

Gestión de memoria

La gestión de memoria en Redox OS utiliza paginación virtual y asignación segura de marcos físicos. El crate `rmm` implementa el gestor de memoria del kernel, garantizando aislamiento entre procesos y eficiencia en la asignación dinámica (OSDev Wiki, 2025a).

Gestión de procesos y multitarea

El kernel de Redox implementa multitarea cooperativa, donde cada proceso se ejecuta en espacio de usuario y el microkernel gestiona la planificación y sincronización básica entre tareas (OSDev Wiki, 2025b).

Sistema de archivos

RedoxFS es el sistema de archivos nativo, completamente escrito en Rust. Ofrece acceso concurrente, soporte para permisos y comunicación mediante el modelo de schemes.

Subsistemas de E/S y drivers

En Redox OS, los controladores de dispositivos se ejecutan como demonios en espacio de usuario. Cada driver tiene su propio namespace restringido, evitando daños al sistema principal. Esta separación aumenta la seguridad y la estabilidad del sistema (Redox OS Project, 2025f).

Comunicación entre módulos

La comunicación se realiza mediante el sistema de schemes. Las llamadas del usuario se traducen en mensajes SQE/CQE, enviados entre el kernel y los servicios de usuario. Esto asegura modularidad y aislamiento total entre componentes (Redox OS Project, 2025a).

Planificación y temporización

El planificador emplea colas de prioridad para asignar tiempo de CPU. Busca mantener equidad, prioridad y escalabilidad con una complejidad cercana a $O(1)$ (OSDev Wiki, 2025b).

Gestión de usuarios y permisos

Los servicios `authd` y `user` manejan autenticación, control de accesos y separación de privilegios, previniendo escaladas no autorizadas (Redox OS Project, 2025h).

Interfaz gráfica (GUI)

`Orbital` y `OrbTK` implementan el entorno gráfico de Redox. Incluyen un servidor de ventanas, gestión de eventos y un toolkit modular, todo en Rust (Redox OS Project, 2025d).

Red y conectividad

El sistema de red, compuesto por `netstack`, `ethernetd` e `ipd`, implementa una pila TCP/IP segura en espacio de usuario, sin requerir acceso directo al kernel (Redox OS Project, 2025b).

Gestión del sistema y servicios

Servicios como `init`, `logd` y `backgroundd` administran la inicialización, el registro de eventos y procesos en segundo plano, similares al sistema `init` de Unix (Redox OS Project, 2025j).

Bibliotecas del sistema y compatibilidad

`relibc` y `redox_rt` ofrecen compatibilidad POSIX. Estas bibliotecas traducen funciones estándar (como `open`, `read`, `write`) a operaciones sobre `schemes` del sistema (Redox OS Project, 2025e).

1.1.7 Herramientas utilizadas

El ecosistema de desarrollo de Redox OS está construido íntegramente sobre las herramientas del lenguaje `Rust` y un conjunto de utilidades de compilación y virtualización que garantizan portabilidad y reproducibilidad en distintos entornos.

Entre las principales herramientas empleadas se destacan:

- **Rust y Cargo:** Redox OS está escrito completamente en `Rust`, utilizando `cargo` como sistema de compilación y gestión de dependencias. Cada componente del sistema (`kernel`, `drivers`, librerías y servicios) se organiza como un *crate*.
- **Make y scripts de automatización:** El proceso de construcción global se gestiona mediante archivos `Makefile` y scripts en `Bash`, que coordinan la compilación cruzada de todos los `crates` y bibliotecas.

- **Podman y Docker:** Se utilizan para entornos de compilación reproducibles. Estas herramientas permiten construir el sistema dentro de contenedores sin necesidad de alterar la configuración del host.
- **QEMU:** Es el emulador principal para ejecutar y depurar Redox OS sin requerir hardware físico. Permite probar el kernel, los controladores y las aplicaciones en un entorno controlado.
- **Git y GitLab:** Todo el código fuente está alojado en GitLab (anteriormente en GitHub). Git se usa para control de versiones, colaboración y revisión de código.
- **Herramientas de compilación cruzada (cross):** Permiten compilar Redox para distintas arquitecturas como `x86_64`, `i686` o `ARM64`, manteniendo el mismo entorno de desarrollo.

Estas herramientas no solo simplifican el desarrollo, sino que también facilitan la experimentación y el aprendizaje, ya que permiten reconstruir completamente el sistema operativo desde el código fuente, probarlo en un entorno virtual y modificar cualquier parte del kernel o de los servicios del usuario.

Corresponde a la sección 6.1–6.8 del manual oficial de Redox OS sobre el proceso de construcción y herramientas de compilación (Redox OS Project, 2025i).

1.1.8 Nivel de complejidad y accesibilidad para estudiantes

Redox OS presenta un equilibrio notable entre complejidad técnica y accesibilidad educativa. Aunque es un sistema operativo funcional y moderno, su diseño modular

en torno a un microkernel y su implementación en **Rust** facilitan el estudio y la modificación de sus componentes por parte de estudiantes o investigadores.

- **Código fuente legible y seguro:** Rust impone reglas de seguridad de memoria y control de concurrencia, reduciendo errores comunes de punteros y sincronización. Esto hace que el código de Redox sea más fácil de entender para quienes se inician en el desarrollo de sistemas.
- **Arquitectura microkernel:** Su estructura simplificada y modular permite analizar cada subsistema de manera aislada (memoria, procesos, archivos, drivers, etc.), lo que favorece la comprensión progresiva del sistema operativo.
- **Facilidad de compilación y pruebas:** Gracias a QEMU y a los contenedores de Podman, los estudiantes pueden compilar y ejecutar Redox en sus propias máquinas sin riesgo de dañar su sistema anfitrión.
- **Documentación educativa:** El Redox OS Book ofrece guías detalladas para compilar, depurar y extender el sistema, convirtiéndolo en una herramienta pedagógica ideal para cursos de sistemas operativos o arquitectura de software.
- **Comunidad activa:** Existe una comunidad abierta de desarrolladores y estudiantes en GitLab y Matrix que brindan soporte, facilitan la colaboración y fomentan el aprendizaje conjunto.

Corresponde a las secciones 1.1–1.7 y 6.1–6.8 del manual oficial de Redox OS (Redox OS Project, 2025c,i).

1.2 FlexOS

1.2.1 Nombre del proyecto o sistema operativo

El sistema operativo analizado es “FlexOS”. Según (Kuenzer, Simon and Lefeuvre, Hugo and Huici, Felipe and others, 2023), su nombre proviene del término “Flexible Operating System”, reflejando su principal objetivo: ofrecer una plataforma operativa flexible y configurable que permita ajustar dinámicamente el nivel de aislamiento entre componentes. A diferencia de los sistemas tradicionales con arquitecturas rígidas (monolíticas o microkernel), FlexOS propone una arquitectura adaptable donde el desarrollador puede equilibrar seguridad, aislamiento y rendimiento según las necesidades del sistema.

1.2.2 Enlace al repositorio y/o documentación oficial

- Repositorio oficial: <https://github.com/project-flexos>
- Sitio y documentación técnica: <https://project-flexos.github.io/>
- Publicación principal: “*FlexOS: Making OS Isolation Flexible*”, presentada en EuroSys 2023 (Kuenzer, Simon and Lefeuvre, Hugo and Huici, Felipe and others, 2023).

1.2.3 Objetivo del proyecto

El objetivo central de FlexOS es redefinir el equilibrio entre rendimiento y aislamiento en los sistemas operativos. Mientras los sistemas tradicionales deben optar entre un kernel monolítico rápido o un microkernel seguro pero más costoso en rendimiento,

FlexOS permite ajustar el nivel de aislamiento de manera configurable. Esto se logra mediante su diseño modular, que permite que cada componente (como controladores, memoria o IPC) se ejecute con distintos niveles de separación temporal y espacial. De esta forma, se busca proporcionar un entorno operativo capaz de adaptarse a distintos contextos: desde sistemas embebidos con recursos limitados, hasta entornos de alta seguridad o investigación académica (Kuenzer, Simon and Lefeuvre, Hugo and Huici, Felipe and others, 2023).

1.2.4 Lenguaje(s) de implementación

FlexOS está implementado principalmente en C y C++, con soporte parcial en Rust para módulos experimentales que requieren mayor seguridad en memoria. Su infraestructura está construida sobre el microkernel Unikraft, al que extiende con nuevas bibliotecas y mecanismos de aislamiento. La elección de C y C++ permite mantener compatibilidad con Unikraft y con componentes de sistemas POSIX existentes, mientras que Rust se emplea en zonas críticas de seguridad o donde la verificación de memoria resulta esencial (Kuenzer, Simon and Lefeuvre, Hugo and Huici, Felipe and others, 2023).

1.2.5 Arquitectura del sistema

FlexOS presenta una arquitectura modular y configurable, inspirada en los principios de los microkernels, pero con un grado de flexibilidad superior. Cada componente puede ejecutarse en un dominio de aislamiento, que define si comparte espacio de direcciones, hilos de ejecución o recursos temporales con otros módulos (Kuenzer,

Simon and Lefeuvre, Hugo and Huici, Felipe and others, 2023). Estos dominios pueden configurarse de tres maneras principales:

El núcleo de FlexOS gestiona la coordinación de estos dominios, la planificación del tiempo de CPU, la comunicación entre módulos (IPC) y el control de errores.

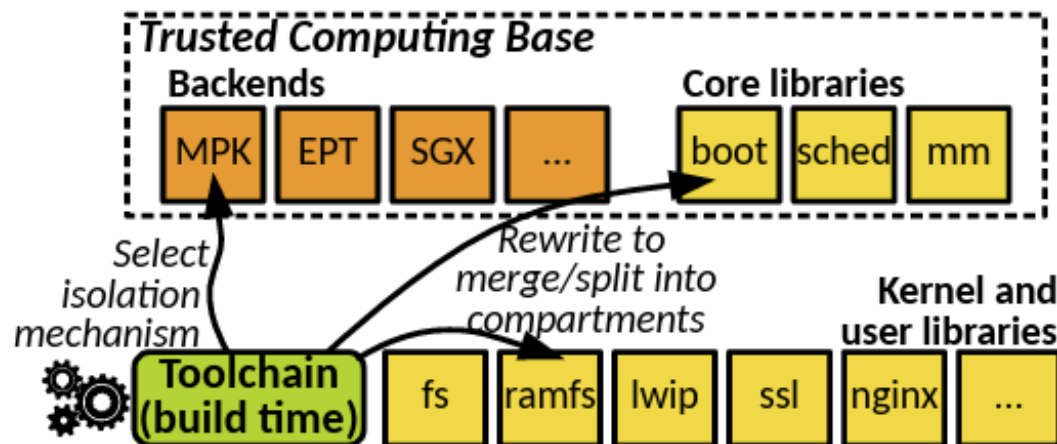


Figura 1.2: Arquitectura modular e híbrida de FlexOS.

Fuente: Obtenido de (Kuenzer, Simon and Lefeuvre, Hugo and Huici, Felipe and others, 2023).

1.2.6 Componentes implementados

En FlexOS, los componentes del sistema se diseñan para maximizar el aislamiento y la modularidad, aplicando principios de sistemas operativos seguros y configurables. Su estructura permite ajustar el nivel de aislamiento (temporal y espacial) entre componentes según los requisitos de rendimiento o seguridad, convirtiéndolo en un sistema operativo híbrido altamente flexible.

Los componentes se implementan principalmente en C, C++ y Rust, utilizando microbibliotecas y servicios del kernel configurables. A continuación, se presenta una tabla con los principales subsistemas y módulos de FlexOS, junto con su función técnica principal (Kuenzer, Simon and Lefeuvre, Hugo and Huici, Felipe and others, 2023).

Tabla 1.2: Funcionamiento técnico de los principales subsistemas de FlexOS

Componente / Sub-sistema	Nombre o módulo en FlexOS	Funcionamiento técnico (idea clave)
Núcleo y planificación	<code>flex_kernel</code>	Controla la ejecución y distribución del tiempo de CPU entre tareas.
Gestión de memoria	<code>mem_mgr</code> , <code>isolation_lib</code>	Maneja memoria virtual y niveles configurables de aislamiento.
Procesos y tareas	<code>task_mgr</code>	Crea y coordina tareas con comunicación segura.
Sistema de IPC	<code>ipc_core</code> , <code>shared_mem</code>	Permite intercambio de datos mediante colas y memoria compartida.
Seguridad y aislamiento	<code>sandbox</code> , <code>trusted_domain</code>	Define dominios seguros y políticas de acceso.
Gestión de E/S y drivers	<code>io_layer</code> , <code>device_mgr</code>	Ejecuta controladores aislados para proteger el kernel.
Monitoreo y fallos	<code>fault_handler</code> , <code>recovery_svc</code>	Detecta errores y reinicia módulos sin afectar el sistema.
Configuración del sistema	<code>flex_config</code>	Ajusta el nivel de aislamiento y rendimiento.
Bibliotecas de usuario	<code>libflex</code> , <code>api_runtime</code>	Facilita la comunicación segura con el kernel.

Fuente: Elaboración propia con base en (Kuenzer, Simon and Lefevre, Hugo and

Huici, Felipe and others, 2023).

Gestión de memoria

FlexOS implementa un modelo de gestión de memoria basado en *espacial isolation*, donde cada dominio o componente puede tener su propio espacio de direcciones. El sistema utiliza tanto protección por hardware (MMU) como políticas de software para garantizar independencia entre procesos y minimizar los efectos de fallos de memoria.

Procesos y multitarea

El administrador de tareas controla el ciclo de vida de los procesos mediante colas de ejecución. Los procesos pueden compartir recursos limitadamente o ejecutarse en dominios completamente aislados, dependiendo de la configuración elegida.

Comunicación entre módulos

FlexOS utiliza un sistema híbrido de comunicación: colas seguras, llamadas IPC síncronas y memoria compartida protegida. Esto permite un equilibrio entre rendimiento y seguridad, ajustable según la política de aislamiento establecida.

Gestión de E/S y drivers

Los controladores en FlexOS se ejecutan como módulos independientes o en dominios restringidos, minimizando el riesgo de fallos. Cada driver puede ser reiniciado o reemplazado sin afectar el núcleo principal.

Seguridad y aislamiento

Una de las características clave del sistema es su capacidad para reconfigurar el aislamiento entre componentes, ofreciendo niveles ajustables de separación espacial y

temporal. Esto permite ejecutar aplicaciones críticas en entornos seguros sin comprometer el rendimiento global del sistema.

1.2.7 Herramientas utilizadas (compiladores, emuladores, etc.)

FlexOS se construye sobre la infraestructura de **Unikraft**, por lo que emplea un conjunto de herramientas especializadas para la compilación, configuración y ejecución de sus módulos. El proceso de construcción del sistema operativo se basa en la cadena de herramientas de GNU Make y Kconfig, lo que permite personalizar la inclusión o exclusión de componentes del kernel según el nivel de aislamiento deseado.

Entre las herramientas más relevantes utilizadas en FlexOS se encuentran:

- **Compilador principal:** clang/LLVM, utilizado para la instrumentación del código, generación de binarios seguros y soporte para aislamiento entre dominios.
- **Infraestructura de construcción:** Unikraft Build System, que proporciona un entorno modular basado en KBuild y Makefiles para definir configuraciones del kernel y librerías.
- **Simuladores y entornos de prueba:** ejecución de instancias de FlexOS sobre QEMU y máquinas virtuales KVM, permitiendo validar configuraciones con diferentes niveles de aislamiento y medir overheads de seguridad.
- **Instrumentación y análisis:** uso de AddressSanitizer (ASan), Clang Control Flow Integrity (CFI) y trazadores de rendimiento para detectar vulnerabilidades y medir costos de comunicación entre dominios.

- **Herramientas de fuzzing y validación:** integración con **ConfFuzz**, el framework de fuzzing desarrollado por los mismos autores, usado para explorar automáticamente configuraciones de aislamiento y detectar fallos en el espacio de diseño.

1.2.8 Nivel de complejidad y accesibilidad para estudiantes

FlexOS presenta un nivel de complejidad intermedio a avanzado, dado su enfoque experimental y su dependencia de la infraestructura de Unikraft y del compilador LLVM. Aunque está diseñado como un sistema de investigación y enseñanza sobre *microkernels híbridos y aislamiento flexible*, su curva de aprendizaje es considerablemente mayor que la de sistemas educativos tradicionales como XV6 o MINIX.

Su accesibilidad depende del objetivo académico:

- Para estudiantes de pregrado, el entorno de FlexOS puede resultar complejo por su uso intensivo de compilación cruzada, configuración Kconfig y dependencias de LLVM, pero es útil para entender conceptos de modularidad y aislamiento.
- Para estudiantes de posgrado o investigadores, ofrece un entorno ideal para estudiar la relación entre *rendimiento y seguridad*, mediante experimentación con distintos niveles de compartimentalización, IPC y aislamiento de memoria.

1.3 FreeDOS

1.3.1 Nombre del proyecto o sistema operativo

El sistema operativo FreeDOS, su nombre proviene de la idea de ofrecer una alternativa libre y abierta al sistema operativo MS-DOS, tras el anuncio de Microsoft en 1994 de interrumpir el soporte a este último (FreeDOS Project, 2025b).

1.3.2 Enlace al repositorio y/o documentación oficial

- Repositorio oficial: <https://github.com/FDOS>
- Sitio web del proyecto: <https://www.freedos.org/>

1.3.3 Objetivo del proyecto

El objetivo principal de FreeDOS es proporcionar un sistema operativo compatible con MS-DOS completamente libre, de código abierto y mantenido por la comunidad. Busca preservar la capacidad de ejecutar aplicaciones y controladores diseñados originalmente para DOS, especialmente aquellas dependientes de entornos de 16 bits (FreeDOS Project, 2025b).

1.3.4 Lenguaje(s) de implementación

FreeDOS está desarrollado principalmente en C, con secciones escritas en x86 Assembly, especialmente en el núcleo y los controladores de dispositivo. El lenguaje C se utiliza para la lógica general del sistema, mientras que el ensamblador proporciona control de bajo nivel sobre interrupciones, rutinas BIOS y gestión de hardware. Este enfoque

híbrido permite combinar eficiencia y compatibilidad con el hardware x86, tal como lo hacía MS-DOS (FreeDOS Project, 2025a).

1.3.5 Arquitectura del sistema

FreeDOS hereda la arquitectura clásica de MS-DOS, organizada en capas que separan el hardware del usuario mediante el BIOS y un núcleo DOS. El sistema se ejecuta en modo real (16 bits), operando directamente sobre la arquitectura Intel x86 sin protección de memoria ni multitarea.

Se presenta un esquema adaptado que representa la arquitectura del sistema operativo MS-DOS en que se basa FreeDOS.

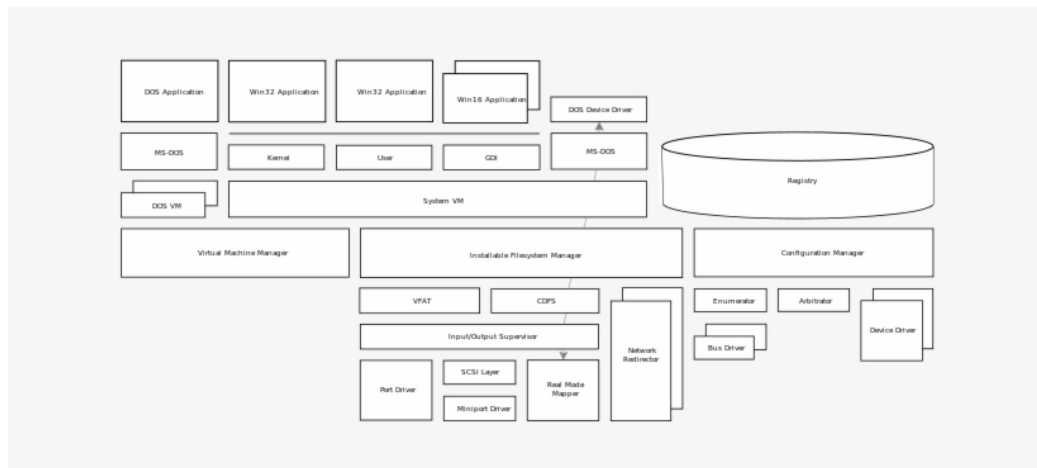


Figura 1.3: Arquitectura clásica de MS-DOS, base de FreeDOS.

Fuente: Adaptado de Microsoft Press, (NicePNG – Image uploader, 2018), *FreeDOS*

Kernel: An MS-DOS Emulator (1996).

Capítulo 2

Comparación técnica

La tabla siguiente presenta una comparación técnica de varios sistemas operativos destacados, considerando aspectos clave como arquitectura, lenguaje de programación, tamaño del kernel, soporte de hardware, documentación y comunidad de usuarios y desarrolladores.

Capítulo 3

Análisis crítico

Referencias

FreeDOS Project (2025a). Developers – The FreeDOS Project. Accedido: 2 noviembre 2025.

FreeDOS Project (2025b). FreeDOS – Sistema operativo libre compatible con DOS. Accedido: 2 noviembre 2025.

Kuenzer, Simon and Lefeuvre, Hugo and Huici, Felipe and others (2023). FlexOS: Making OS Isolation Flexible. In *Proceedings of the 18th EuroSys Conference*. ACM. Accedido: 26 octubre 2025.

NicePNG – Image uploader (2018). Architectural Diagram – MS-DOS Architecture Diagram. Accedido: 02 noviembre 2025.

OSDev Wiki (2025a). Memory Management – OSDev Wiki. Accedido: 25 octubre 2025.

OSDev Wiki (2025b). Scheduling Algorithms – OSDev Wiki. Accedido: 26 octubre 2025.

Redox OS Project (2025a). Communication – The Redox Operating System Book. Accedido: 26 octubre 2025.

Redox OS Project (2025b). Components of Redox – The Redox Operating System Book. Accedido: 26 octubre 2025.

Redox OS Project (2025c). Documentación oficial de Redox OS. Accedido: 22 octubre 2025.

Redox OS Project (2025d). Graphics and Windowing – The Redox Operating System Book. Accedido: 26 octubre 2025.

Redox OS Project (2025e). Programs and Libraries – The Redox Operating System Book. Accedido: 26 octubre 2025.

Redox OS Project (2025f). Repositorio de Drivers de Redox OS en GitLab. Accedido: 26 octubre 2025.

Redox OS Project (2025g). Repositorio de Redox OS en GitHub. Accedido: 22 octubre 2025.

Redox OS Project (2025h). Security – The Redox Operating System Book. Accedido: 26 octubre 2025.

Redox OS Project (2025i). The Build Process – The Redox Operating System Book. Accedido: 26 octubre 2025.

Redox OS Project (2025j). User Space – The Redox Operating System Book. Accedido: 26 octubre 2025.